koroteevmv / ML_course
Code
Issues 2
Pull requests
Actions
Projects
Security and quality
Insights
Type / to search
Files
main
Go to file
ML4.1 metrics
data
README.md
ML4.2 threshold
ML4.3 cross-validation
ML4.4 diagnostics
ML4.5 hyperparameters
ML5.1 data integration
ML5.2 numeric features
ML5.3 categorical features
ML5.4 target
ML5.5 EDA
ML6.1 clustering
ML6.2 pca
ML7.3 text
ML7.4 images
misc
LICENSE
ML_course / ML4.1 metrics /
Add file
koroteev-test Картинки распределены по папкам
7dd1f73 · 2 years ago
History
Name
Last commit message
Last commit date
..
data
Картинки распределены по папкам
2 years ago
README.md
Картинки распределены по папкам
2 years ago
README.md

Метрики эффективности моделей машинного обучения

Цель работы

Научиться измерять эффективность моделей машинного обучения с помощью метрик, вибирать метрики исходя из задачи, разбивать датасет на обучающую и тестовую подвыборки.

Содержание работы

1. Загрузите данные о вероятности развития сердечного приступа, прилагающийся к этой работе (heart.csv).
2. Обучите на этих данных простую модель логистической регрессии и выведите метрику точности (accuracy).
3. Разделите датасет на две части - первую половину используйте для обучения, а вторую - для оценки точности. Сравните значения метрик.
4. Разделите датасет на две части случайным образом. Повторите анализ.
5. Разделите датасет с помощью библиотечной функции. Повторите анализ несколько раз.
6. Постройте матрицу классификации и отчет о классификации для обученной модели для обучающей и тестовой выборок. Проинтерпретируйте полученные значения.
7. Подсчитайте для построенной модели значение всех метрик эффективности классификации на тестовой и обучающей выборках. Нужно использовать следующие метрики: accuracy, precision, recall, f1.

Методические указания

После чтения файла вы должны увидеть примерно такую таблицу при выводе первых строк датасета:

index	age	sex	cp	trtbps	chol	fbs	restecg	thalachh	exng	oldpeak	slp	caa	thall	output
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1

Как всегда выделим целевую переменную, сразу же импортируем и обучим модель логистической регрессии:

```
y = data["output"]
x = data.drop("output", axis=1)

from sklearn.linear_model import LogisticRegression

logistic = LogisticRegression().fit(x, y)
logistic.score(x, y)
```

Данная модель показывает более 85% точности. Но эта оценка качества модели является завышенной (оптимистичной), так как она оценена именно по тем данным, на которых модель училась. Чтобы оценить именно обобщающую способность модели, то есть ее способность правильно предсказывать значение целевой переменной для тех объектов, которые она не учитывала при подстройке параметров, необходимо оценить модель на другой выборке. Для этого нужно разделить исходный датасет на две части. Обучающая выборка (первая часть) будет использоваться для нахождения оптимальных значений внутренних параметров модели, а тестовая выборка (вторая часть) - для оценки качества полученной модели.

Разделение выборки

Делить датасет можно разными способами. Важно, чтобы каждый конкретный объект попал только в одну выборку - либо тестовую, либо обучающую. То есть эти части должны быть непересекающиеся. Самый простой способ - просто взять какое-то количество объектов в начале датафрейма в обучающую выборку, а остальные - в тестовую. То есть мы просто берем несколько первых строчек (например, 200) для обучающей выборки:

```
x_train, y_train = x[:200], y[:200]
```

Важно убедиться, что все в порядке с формами получившихся массивов:

```
x_train.shape, y_train.shape
```

Итак, первые 200 строк попали в обучающую выборку. Тогда тестовую выборку составять оставшиеся строки датасета:

```
x_test, y_test = x[200:], y[200:]
x_test.shape, y_test.shape
```

Получилось, что в тестовой выборке осталось 103 объекта. Теперь можно заново обучить модель классификации и оценить ее качество. Обратите внимание, что мы вызываем метод *fit()* именно на обучающей части датасета. А вот эффективность измеряем сначала на обучающей части, а затем на тестовой, чтобы сравнить:

```
logistic_test = LogisticRegression().fit(x_train, y_train)
logistic_test.score(x_train, y_train), logistic_test.score(x_test, y_test)
```

Получилось, что на обучающей выборки точность модели даже немного повысилась, до 90%. Это произошло за счет того, что в обучающей выборке меньше данных, чем в целом датасете. А чем меньше точек, тем проще модели под них подстроиться. А вот эффективность модели на тестовых данных стала сильно ниже - всего 54%. Это значит, что наша модель чуть лучше, чем простое угадывание. Хотя, как увидим дальше, на самом деле не все так плохо.

Вместо того, чтобы вручную задавать количество объектов, которые отводятся на обучающую выборку, можно выразить это количество через процент от всего объема датасета. Это даст нам универсальность:

```
N = int(x.shape[0] * 0.8)

x_train, y_train, x_test, y_test = x[:N], y[:N], x[N:], y[N:]
x_train.shape, y_train.shape, x_test.shape, y_test.shape
```

В данном примере мы взяли 80% объектов для обучающей выборки и 20% - оставили на тестовую. Это довольно стандартная схема разбиения и мы в общем случае будем придерживаться именно ее. Вот какие формы получились у массивов:

```
((242, 13), (242,), (61, 13), (61,))
```

Давайте еще раз обучим и оценим модель, уже на новом разделении:

```
logistic_test = LogisticRegression().fit(x_train, y_train)
logistic_test.score(x_train, y_train), logistic_test.score(x_test, y_test)
```

Мы видим, что точность на обучающей выборке (мы ее будем называть обучающей точностью) чуть опустилась - опять же это эффект количества точек - сейчас их чуть больше. Но и тестовая эффективность (точность модели, измеренная на тестовой выборке) тоже подросла - до 62%. Это потому, что чем больше примеров мы использовали для обучения, тем более качественной и обобщающей получается наша модель.

Но разница между обучающей и тестовой точностью у нашей модели все еще довольно высока. С чем это может быть связано? На самом деле, мы поступаем немного неправильно. Нам нужно более внимательно присмотреться к данным. Мы уже видели первые несколько строк датасета. Давайте еще выведем последние несколько строк:

```
data.tail()
```

index	age	sex	cp	trtbps	chol	fbs	restecg	thalachh	exng	oldpeak	slp	caa	thall	output
298	57	0	0	140	241	0	1	123	1	0.2	1	0	3	0
299	45	1	3	110	264	0	1	132	0	1.2	1	0	3	0
300	68	1	0	144	193	1	1	141	0	3.4	1	2	3	0
301	57	1	0	130	131	0	1	115	1	1.2	1	1	3	0
302	57	0	1	130	236	0	0	174	0	0.0	1	1	2	0

Можно заметить, что вы первой половине данных целевая переменная всегда положительная, а во второй - отрицательная. Другими словами, датасет отсортирован по значению целевой переменной. Поэтому наше разбиение имеет один неприятный эффект - в обучающей выборке скапливается большое количество положительных примеров, а в тестовой - только отрицательные. Другими словами, обучающая и тестовая выборки очень непохожи друг надруга. А для того, чтобы наше оценивание сработало, нужно, чтобы они были как можно более однородными.

Поэтому просто выбрать первые строки датасета обычно не очень рациональное решение. Даже если датасет не отсортирован по значению целевой переменной, он вполне может быть упорядочен по какому-то другому признаку, или хотя бы по времени добавления объектов в набор или базу данных. А такое упорядочение вносит систематическую ошибку выборки, которая смещает оценку качества модели.

Из-за этого делить данные на обучающую и тестовую выборки практически всегда нужно случайным образом. То есть выбрать случайное подмножество точек и поместить их в обучающий набор, а оставшиеся - в тестовый. Основная техническая трудность здесь состоит в том, что датасет у нас разделен на две переменные - матрицу признаков и вектор значений целевой переменной. Просто воспользоваться методом случайного выбора из массива мы не можем, так как когда мы повторим его два раза - для переменных *x* и *y* у нас будут выбраны не соответствующие друг другу объекты.

Для решения этой проблемы воспользуемся индексными масками. Заведем массив булевских значений, в которых количество истинных значений будет соответствовать желаемому объему обучающей выборки, а ложных - объему тестовой:

```
mask = np.array([True] * N + [False] * (y.shape[0] - N))
```

В сумме должно получиться в точности количество объектов в полном датасете.Теперь перемешаем этот массив, используя стандартную функцию *numpy*:

```
from numpy.random import shuffle

shuffle(mask)
mask
```

```
array([False,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True, False, False,  True,  True,
        True,  True,  True,  True,  True, False,  True,  True,  True,
        ...
        True, False,  True,  True, False,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True])
```

Этот булев массив можно использовать как маску при индексировании исходного датасета. Если выбрать данные по этой маске, то в итог попадут только те элементы, которые стоят на тех местах, на которых в маске - истина:

```
x_train = x[mask]
x_train.shape
```

Смысл этого приема в том, что мы можем применить одну и ту же маску к обоим частям массива. А если инвертировать эту маску, то выберем оставшиеся значения:

```
x_train, y_train, x_test, y_test = x[mask], y[mask], x[~mask], y[~mask]
x_train.shape, y_train.shape, x_test.shape, y_test.shape
```

После такого разделения опять обучим и оценим модель:

```
logistic_test = LogisticRegression().fit(x_train, y_train)
logistic_test.score(x_train, y_train), logistic_test.score(x_test, y_test)
```

Мы видим, что теперь разница между тестовой и обучающей выборкой стала гораздо меньше - 88% и 77% соответственно. Это как раз обосновано более правильным разделением, которое дает однородные по своему составу части датасета.

Конечно, такая стандартная процедура как случайное разделение датасета на две части уже реализована в виде готовой функции - в реальной работе не придется каждый раз дублировать приведенный код. Такая функция лежит в пакете *sklearn*, который отвечает за методологию выбора модели - обэтом мы поговорим в следующих работах:

```
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(x, y, train_size=0.8)
x_train.shape, y_train.shape, x_test.shape, y_test.shape
```

Обратите внимание, что библиотечная функция возвращает элементы в строго определенном порядке. Его придется либо запомнить, либо пользоваться готовыми сниппетами кода.

Построение метрик качества классификации

До сих пор мы оценивали модель только по одной метрике эффективности. Но для более полного анализе этого недостаточно. В пакете *metrics* собрано множество функций, которые позволяют исследовать поведение уже обученных моделей, в том числе - метрики эффективности. Импортируем несколько нужных нам инструментов:

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report
```

Для использования большинства метрик необходимо передавать в эти функции два вектора - вектор истинных значений целевой переменной и вектор предсказанных значений. Истинные (эмпирические) значения - это часть исходного датасета. А предсказанные (теоретические) значения можно вычислить. Рассчитаем предсказанные значения отдельно для обучающей и для тестовой выборки:

```
y_test_pred = logistic_test.predict(x_test)
y_train_pred = logistic_test.predict(x_train)
```

Первым делом построим матрицу классификации::

```
confusion_matrix(y_train, y_train_pred)
```

Она показывает, как часто и в каких классах модель ошибается. В идеальной модели ненулевые значения должны быть только на главной диагонали:

```
array([[ 80,  30],
       [ 11, 121]])
```

Обратите внимание, что для правильного анализа модели нужно строить все эти диагностические инструменты именно по тестовой выборке:

```
confusion_matrix(y_test, y_test_pred)
```

```
array([[23,  5],
       [ 0, 33]])
```

Именно эти данные покажут истинное поведение модели.

Еще один полезный инструмент - отчет о классификации:

```
print(classification_report(y_test, y_test_pred))
```

```
              precision    recall  f1-score   support

           0       1.00      0.82      0.90        28
           1       0.87      1.00      0.93        33

    accuracy                           0.92        61
   macro avg       0.93      0.91      0.92        61
weighted avg       0.93      0.92      0.92        61
```

Он показывает всю основную информацию по итогам классификации. Познакомьтесь со структурой данного отчета и проинтерпретируйте полученные результаты.

Что касается остальных метрик качества классификации, можно строить их отдельно:

```
precision_score(y_test, y_test_pred)
```

Но более удобно свести их все в таблицу при помощи датафрейма:

```
metrics = pd.DataFrame({
    "Train": [
        accuracy_score(y_train, y_train_pred),
        precision_score(y_train, y_train_pred),
        recall_score(y_train, y_train_pred),
        f1_score(y_train, y_train_pred),
    ],
    "Test": [
        accuracy_score(y_test, y_test_pred),
        precision_score(y_test, y_test_pred),
        recall_score(y_test, y_test_pred),
        f1_score(y_test, y_test_pred),
    ],
}, index = ["Accuracy", "Precision", "Recall", "F1"])

metrics
```

index	Train	Test
Accuracy	0.8305785123966942	0.9180327868852459
Precision	0.8013245033112583	0.868421052631579
Recall	0.9166666666666666	1.0
F1	0.8551236749116607	0.9295774647887324

Здесь мы видим сравнение четырех разных основных метрик качества модели классификации по обучающей и по тестовой выборкам. Видно, что тестовые метрики все чуть ниже, чем обучающие. Это довольно типичное поведение моделей машинного обучения. Мы уже рассматривали причины этого. Но помните, что это не гарантируется и в конкретном случае, тестовые метрики могут быть как очень близки, так и даже выше. Все решает случай при рандомном разеделении выборки.

При оценке эффективности модели машинного обучения ориентирвоаться нужно только на тестовые метрики. Но сравнение уровня тестовой и обучающей метрики - это один их основных инструментов диагностики модели машинного обучения.

Задания для самостоятельного выполнения

1. Повторите анализ для других видов моделей. Используйте 5-10 разных классов моделей. Подсчитывайте только метрики на тестовой выборке.
2. Повторите анализ для другого датасета по вашему выбору. Используйте несколько моделей для сравнения. Используйте датасет для множественной классификации.
3. Повторите анализ для датасета, предназначенного для решения задачи регрессии. Используйте все метрики качества регрессии, изученные на лекции. Постройте 5 - 10 разных моделей регрессии.

Контрольные вопросы

1. Почему для анализа модели нужно применять несколько метрик эффективности?
2. Зачем для анализа качества модели делить датасет на обучающую и тренировочную выборки?
3. В чем особенность и область применения каждой метрики качества?
4. Для каждой метрики классификации и регрессии придумайте пример, в котором данная метрика будет определяющей для выбора наилучшей модели.
5. В каких случаях нельзя делить выборку случайным образом?
6. Зачем нужен и как использовать отчет о классификации

Дополнительные задания

1. Постройте график зависимости метрик качества классификации из методических указаний от процента выборки, отводимого на тестовую часть.
2. Создайте универсальную функцию, которая подсчитывает все метрики по любой модели и выводит их в виде таблицы.
3. Создайте функцию, принимающая на вход датасет, которая проверяет наибольшее количество разных моделей машинного обучения и выдающая датафрейм с таблицей всех метрик.